Assignment 4, Stochastic optimization

B: Log-Logistic Dose-Response Curves

Carl-Emil Krarup Jensen

University of Copenhagen
Department of Mathematical Sciences

November 4, 2026

Theoretical background

We consider the four-parameter log-logistic model

$$f(x;\theta) = \gamma + \frac{\delta - \gamma}{1 + \exp(\beta \log x - \alpha)}, \qquad \theta = (\alpha, \beta, \gamma, \delta)^\top.$$

We consider observations of the form

$$Y_i = f(x_i;\theta) + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

We estimate θ by minimizing the mean squared error:

$$H(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - f(x_i;\theta)\right)^2.$$

Theoretical background

Write $z_i = \log x_i$, $q_i = (1 + \exp(\beta z_i - \alpha))^{-1}$, and $r_i = f(x_i; \theta) - y_i$.
The gradient of the mean response is

$$J_i = \nabla_\theta f(x_i; \theta) = \begin{pmatrix} (\delta - \gamma) q_i (1 - q_i) \\ -(\delta - \gamma) z_i q_i (1 - q_i) \\ 1 - q_i \\ q_i \end{pmatrix}.$$

Thus,

$$\nabla H(\theta) = \frac{2}{N} \sum_{i=1}^{N} r_i J_i,$$

$$\nabla^2 H(\theta) = \frac{2}{N} \sum_{i=1}^{N} \left[J_i J_i^\top + r_i \nabla_\theta^2 f(x_i; \theta) \right].$$

- $J_i J_i^\top$ is positive semidefinite, but the residual term can make the Hessian indefinite.
- The objective is generally nonconvex; starting values can affect convergence.

Theoretical background

For any direction v,

$$v^\top \nabla^2 H(\theta) v = \underbrace{\frac{2}{N} \sum_i (J_i^\top v)^2}_{\geq 0} + \underbrace{\frac{2}{N} \sum_i r_i \, v^\top \nabla_\theta^2 f(x_i; \theta) v}_{\text{can be negative}} .$$

Since the residuals r_i can have either sign If the second term outweighs the first, then

$$v^\top \nabla^2 H(\theta) v < 0.$$

Consequences for the algorithms:

- Different starting values can lead to different stationary points or nearly flat regions.
- GD moves downhill locally; SGD follows noisy gradients. Neither is guaranteed to find a global minimum.

Theoretical background

We will be comparing

- Full batch gradient descent
- Stochastic gradient descent
- Polyak momentum
- Nesterov acceleration
- Adaptive gradient

Theoretical background

Move in the negative gradient direction, using all observations.

Algorithm 1 Full-batch gradient descent

Choose initial coefficients $\theta_{\texttt{init}}$
for $i = 0, \ldots, k-1$ **do**
    $g_i \leftarrow \nabla H(\theta_i)$
    if $\|g_i\|_\infty < \varepsilon$ **then**
        break
    end if
    $d_i \leftarrow -g_i$
    Choose γ_i by Armijo backtracking
    $\theta_{i+1} \leftarrow \theta_i + \gamma_i d_i$
end for

k: maximum iterations; ε: gradient tolerance.

Theoretical background

Armijo backtracking is used to choose the stepsize for full batch gradient descent.
Given a descent direction d_i, with $g_i^\top d_i < 0$:

1. Start with $\gamma_i = 1$.
2. Accept the step if

$$H(\theta_i + \gamma_i d_i) \leq H(\theta_i) + c\,\gamma_i g_i^\top d_i.$$

3. Otherwise, halve γ_i and try again.

We set $c = 10^{-4}$. The implementation allows at most 60 attempts.

Algorithm 2 SGD with random reshuffling

Choose initial coefficients $\theta_{\texttt{init}}$
Initialize update tracking $i \leftarrow 0$
for $e = 1, \ldots, k$ **do**
    Draw a random permutation π_e of $\{1, \ldots, N\}$
    split permutation into mini-batches of size m
    Set the epoch learning rate $\gamma_e \leftarrow \frac{\gamma_0 K}{K+(e-1)^a}$
    for For each mini batch B_j, $j = 1, \ldots, \lceil N/m \rceil$ **do**
        $i \leftarrow i + 1$
        $g_{i-1} \leftarrow |B_j|^{-1} \sum_{k \in B_j} 2 r_k(\theta_{i-1}) J_k(\theta_{i-1})$
        $\theta_i \leftarrow \theta_{i-1} - \gamma_e g_{i-1}$
    end for
end for
return θ

SGD reduces the cost of each parameter update. A full epoch still processes the entire dataset, so whether it reaches a useful solution faster must be assessed through convergence against elapsed time.

Theoretical background

Polyak momentum incorporates memory of previous gradients by updating the direction via a convex combination:

1. $g_{i-1} \leftarrow |B_j|^{-1} \sum_{k=1}^{|B_j|} 2r_k(\theta_{i-1}) J_k(\theta_{i-1})$
2. $\rho_i \leftarrow \lambda \rho_{i-1} + (1-\lambda) g_{i-1}, \qquad \lambda \in [0, 1)$
3. $\theta_i \leftarrow \theta_{i-1} - \gamma_e \rho_i$

We fix $\lambda = 0.9$ as recommended in [Hansen and Larsson, 2026] throughout the further analysis.
Nesterov acceleration uses the previous movement of the parameter to update

1. $z_{i-1} \leftarrow \theta_{i-1} + b_n(\theta_{i-1} - \theta_{i-2})$
2. $g_{i-1}(z_{i-1}) \leftarrow |B_j|^{-1} \sum_{k \in B_j} 2r_k(z_{i-1}) J_k(z_{i-1})$
3. $\theta_i \leftarrow z_{i-1} - \gamma_e g_{i-1}(z_{i-1})$

We set $b_n = 0.9$ for all n. Because our objective is generally nonconvex, we can't rely on the Nesterov optimal weights. We therefore investigate a practical variant with fixed momentum and an epoch-dependent learning-rate schedule.

Learning-rate decay

We use the power-law learning rate schedule

$$\gamma_e = \frac{\gamma_0 K}{K + (e-1)^a}$$

where e is the number of completed epochs.

- γ_0: initial learning rate.
- a: controls the eventual decay rate.
- K: controls how quickly the rate decreases.

We choose a target rate $\gamma_h = \gamma_0/2$ after h completed epochs. Using the course formula,

$$K = \frac{h^a(\gamma_0/2)}{\gamma_0 - \gamma_0/2}.$$

The learning rate is thus fixed within each epoch.

Algorithm 3 AdaGrad

1: **for** each update $n = 1, 2, \ldots$ **do**
2: Evaluate the current observation or mini-batch gradient g_n at θ_{n-1}
3: $G_n \leftarrow G_{n-1} + \text{diag}(g_{n,1}^2, \ldots, g_{n,4}^2)$
4: $\theta_n \leftarrow \theta_{n-1} - \gamma(\varepsilon I_4 + G_n)^{-1/2} g_n$
5: **end for**

The learning rate parameter γ is kept fixed

However the effective learning rate is

$$\gamma_{n,j}^{\text{eff}} = \frac{\gamma}{\sqrt{\varepsilon + \sum_{t=1}^{n} g_{t,j}^2}}.$$

Coordinates with larger accumulated squared gradients receive smaller rates. This may prevent overshooting the minimum

Numerical comparisons

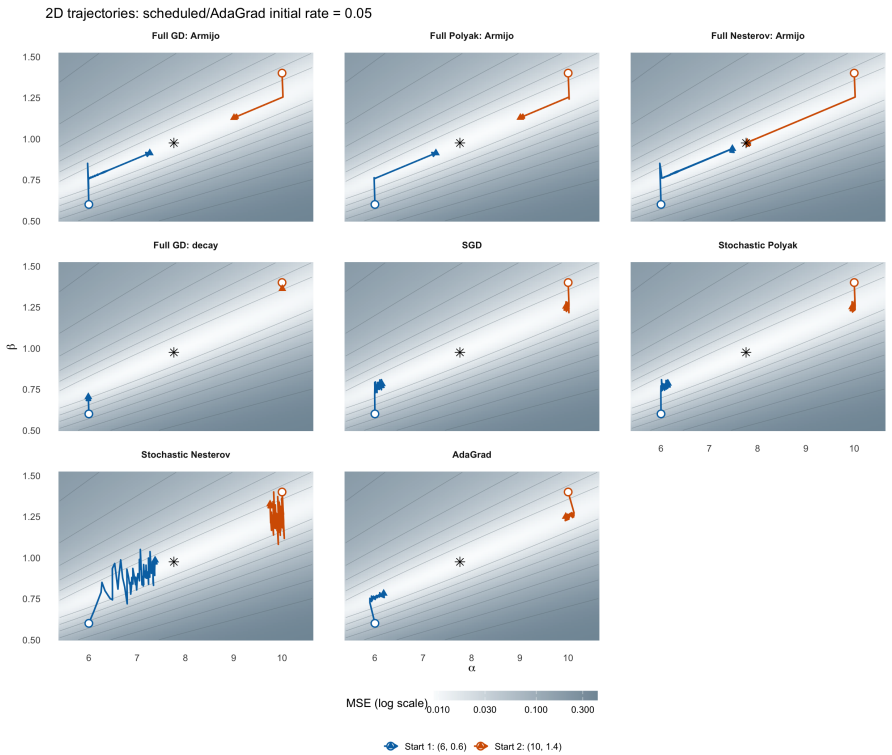
A thorough numerical comparison is very comprehensive and each parameter can be studied in detail by itself. To limit ourselves and to preserve consistency we fix the parameters to the following values unless else is specified:

- Initial learning rate for the decay schedule algorithms $\gamma_0 = 0.05$, and adgrad. $\gamma_0 = 1$ for armijo backtracking
- Initial parameters $(\alpha, \beta, \gamma, \delta) = (6, 0.8, 0.1, 1.3)$
- Armijo backtracking constant $c = 10^{-4}$
- Batch size $|B| = 1$
- We use 15 epochs
- Decay schedule asymptotic exponent $a = 0.75$ and half life $h = 5$
- The momentum parameter $\lambda = 0.9$
- We set the tolerance for stopping deterministic algorithms 10^{-4}, and $\varepsilon = 10^{-8}$ in the adaptive gradient algorithm.

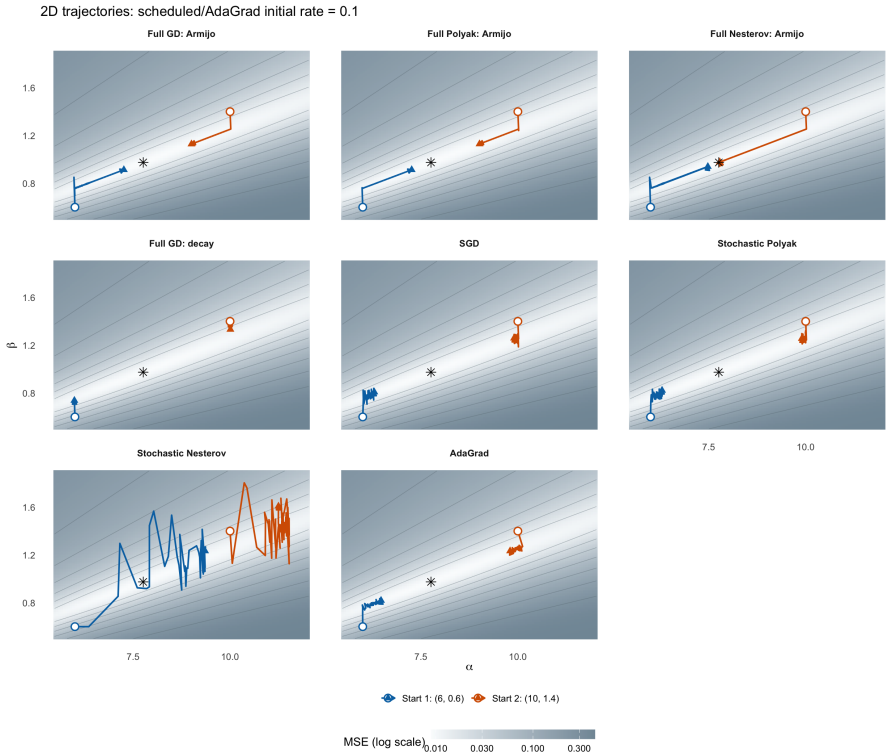
We will mainly examine how the algorithms compare to configurations of N, γ_0, $\theta_{\texttt{init}}$ and θ_0.

Numerical comparisons

Simulations from $\log(x) \sim \mathcal{N}(0, 2^2)$ with $N = 300$ and $(\alpha, \beta, \gamma, \delta) = (6, 0.8, 0.1, 1.3)$.



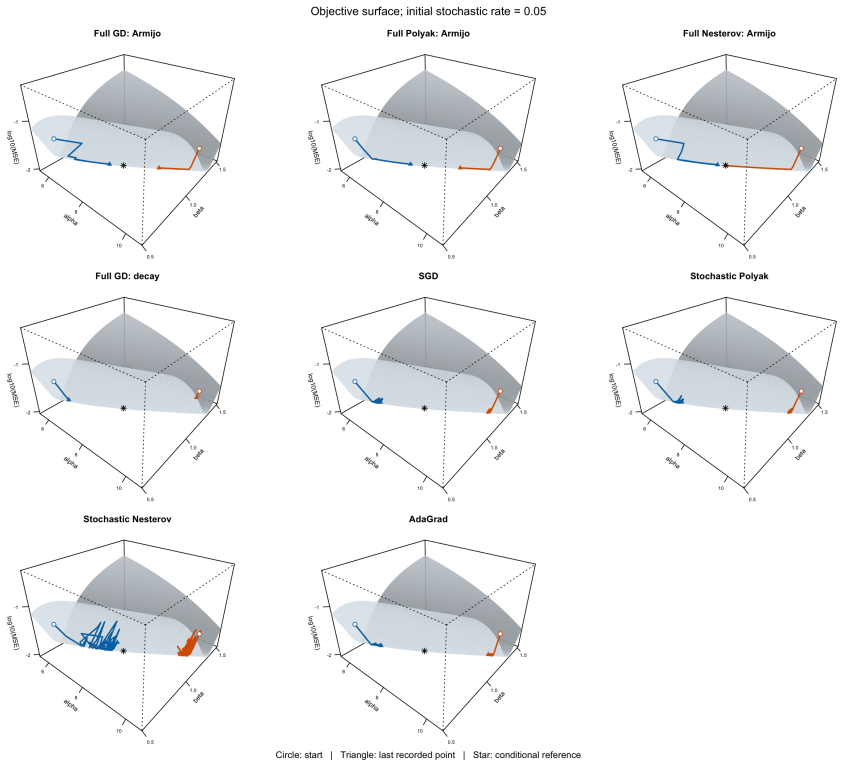
$\gamma_0 = 0.05$



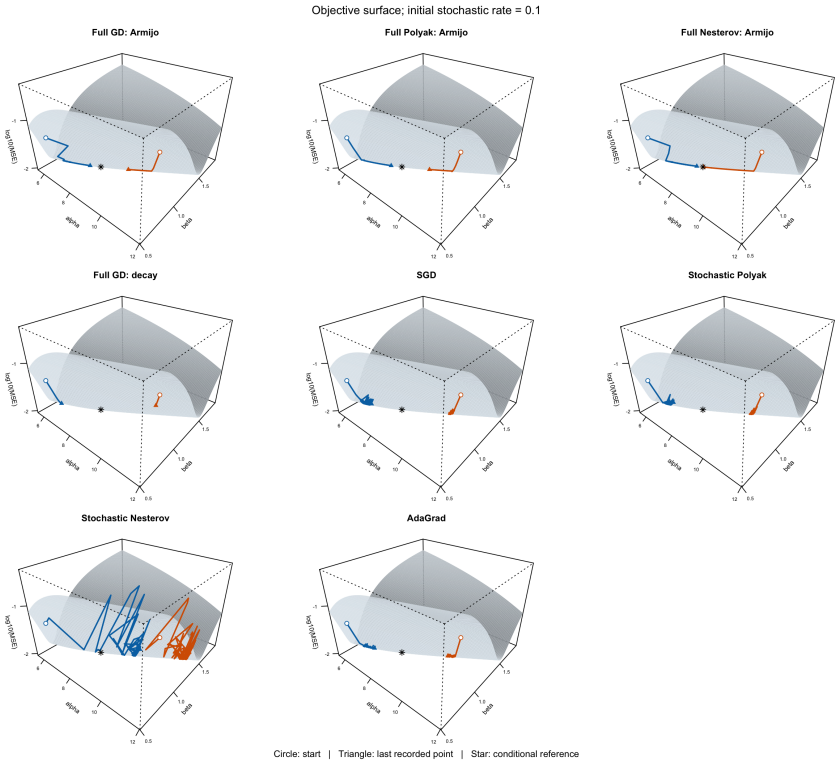
(b) $\gamma_0 = 0.1$

Numerical comparisons

Simulations from $\log(x) \sim\sim \mathcal{N}(0, 2^2)$ with $N = 300$ and $(\alpha, \beta, \gamma, \delta) = (6, 0.8, 0.1, 1.3)$.



$\gamma_0 = 0.05$

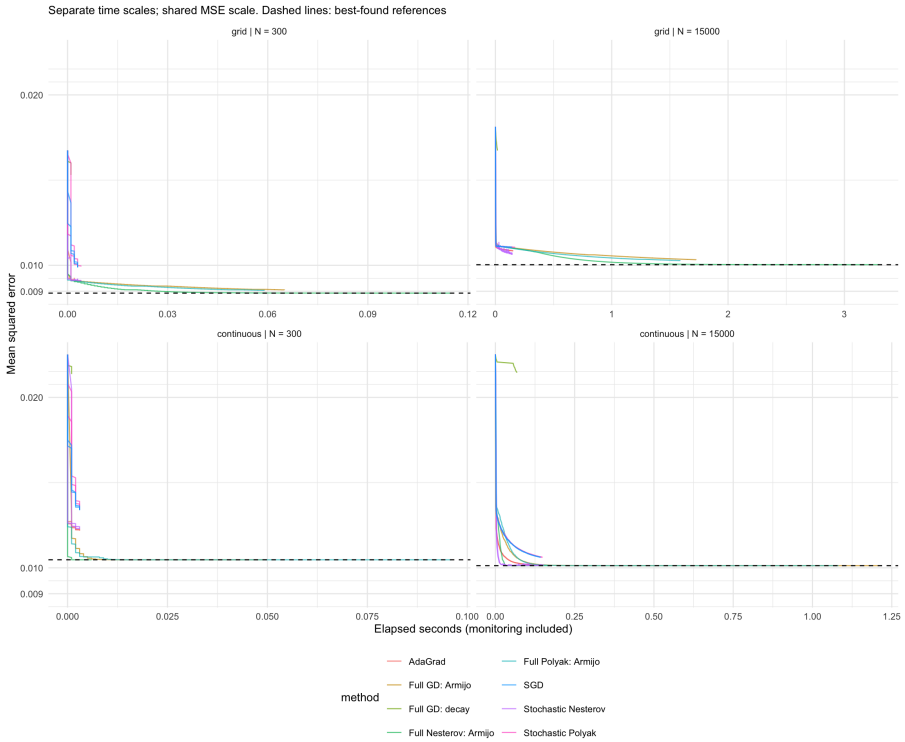


(b) $\gamma_0 = 0.1$

Numerical comparison

Larger simulation study with $N = 300$ and $N = 15000$: Samples are drawn in two ways: From exponential grid points $e, e^2, \ldots, e^{15}$, and from a continuous distribution $\log(x) \sim \mathcal{N}(0, 2^2)$.

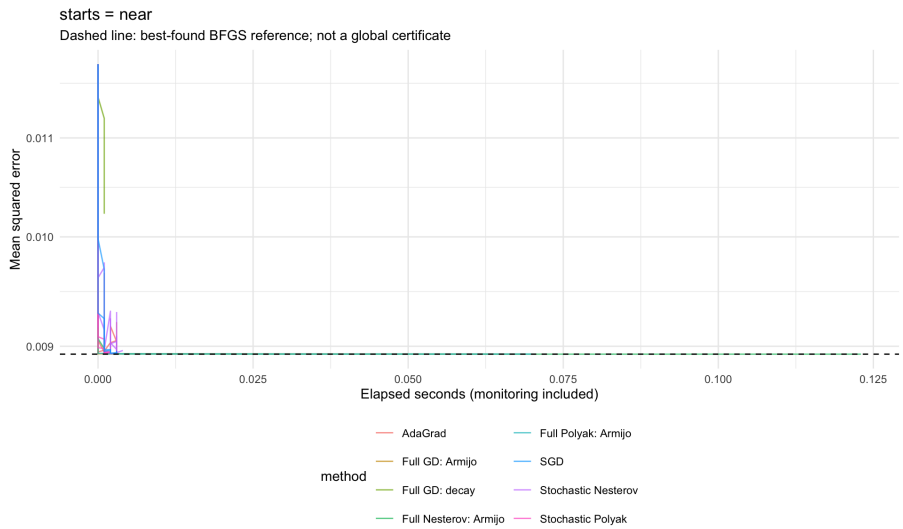
- The stochastic algorithms reduce the loss fast.
- When the loss decrease is small, deterministic algorithms show better capability of reducing the error.
- In short time horizons stochastic algorithms show comparable performance to the deterministic algorithms.



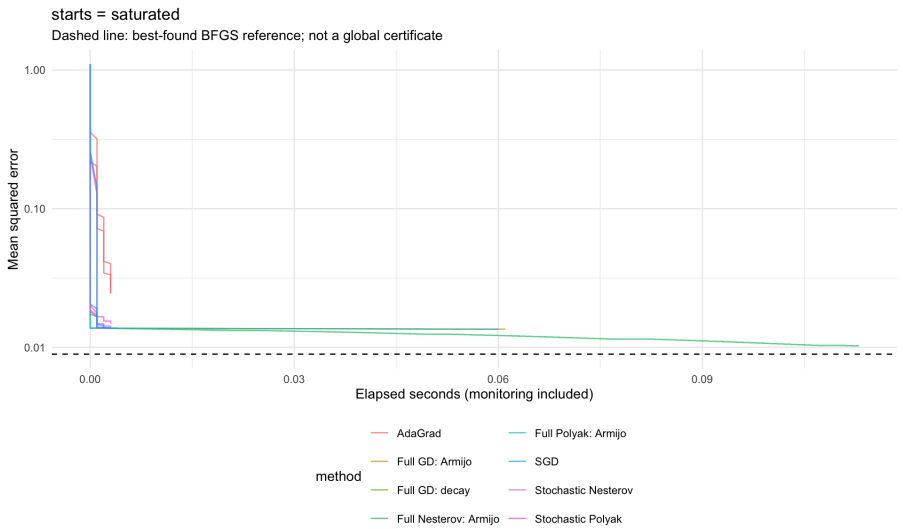
MSE and elapsed time for default parameters

Numerical comparison

What if we take a poor initial guess[1]?
We set two distinct starting points for the algorithms: One near the truth $(7.5, 0.9, 0.15, 1.25)$, and one distant from the truth $(20, 2, 0.5, 2.3)$.



(a) Initial guess close to the truth



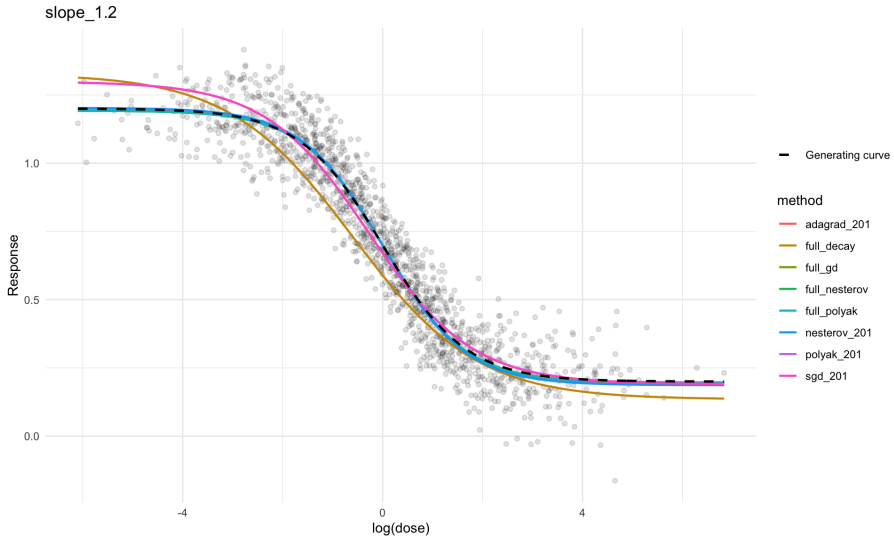
(b) Initial guess far from the truth

All algorithms show robustness to poor initial guesses, with adagrad showing the worst performance under the poor initial guess. While Full GD with decay performs the worst when the guess is close to the truth.

[1] These simulations were performed with $N = 300$ on the exponential grid

Numerical comparison

What if we vary the conditioning of the problem[2]?
We keep $(\alpha, \gamma, \delta) = (0, 0.2, 1.2)$ and vary $\beta \in \{0.35, 1.2, 4\}$. The problem isn't strongly convex, however we may use relative difference between the largest eigenvalue and the smallest eigenvalue of the hessian $\lambda_{max}/\lambda_{min}$ to understand the curvature.



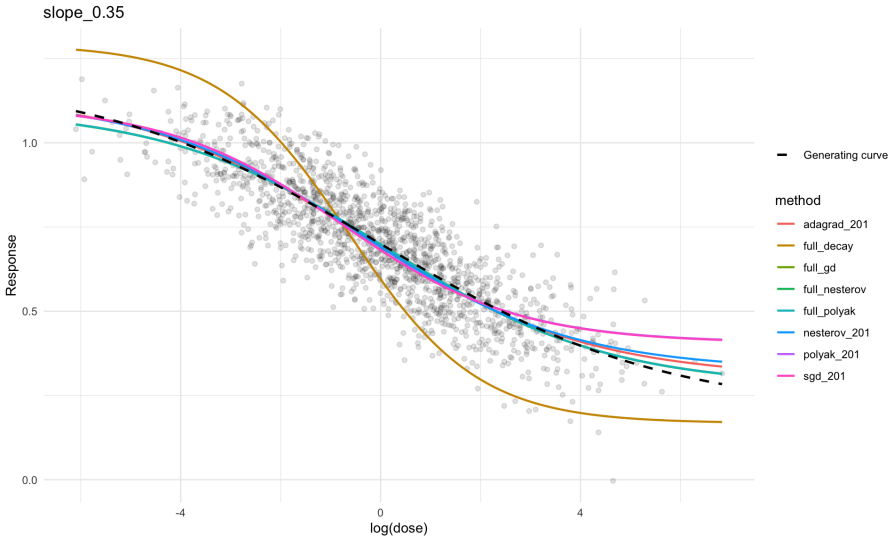
(a) Data generating curve and fits with $\beta = 1.2$

True β	$\frac{\lambda_{max}}{\lambda_{min}}$
0.35	493
1.2	103
4	970

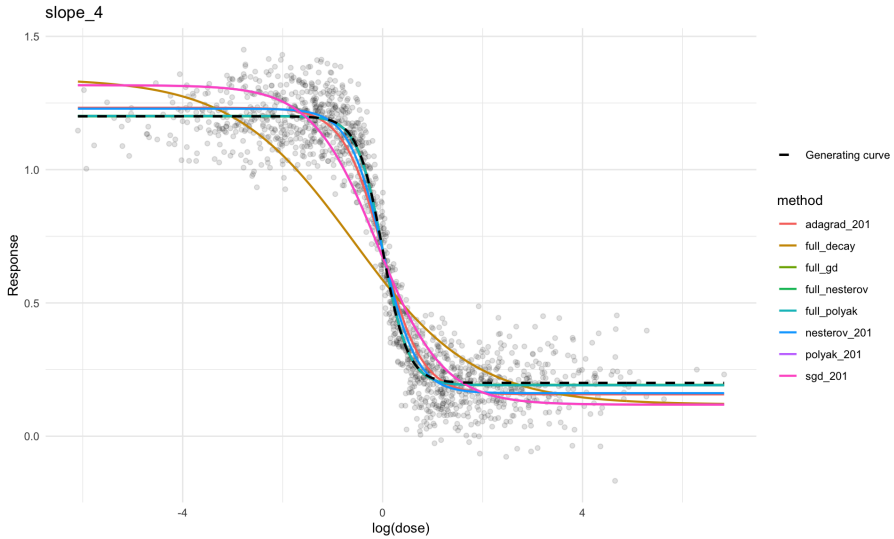
[2]These simulations were performed with $N = 1500$ log doses sampled from the $\mathcal{N}(0, 2^2)$ distribution. We keep the initial guess fixed so the distance to the truth will also vary across experiment.

Numerical comparison

Full gradient descent with decay scheduling doenst converge. Vanilla stochstic gradient descent struggles to fit the tails when λ_{max} is big.



(a) Data generating curve and fits with $\beta = 0.35$



(a) Data generating curve and fits with $\beta = 4$

Implementation: Choices, tests and construction

The project has been structured as follows.

Project structure

```
assignment_4
├── project_B
│   ├── data.R
│   ├── experiment.R
│   ├── gradient.cpp
│   ├── model.R
│   ├── plots.R
│   └── solvers.R
├── report
└── tests/testthat
    ├── test-core.R
    └── test-cpp.R
```

Example: `model.R`

```
gradient <- function(theta, problem, index = NULL) {
  z <- problem$z
  y <- problem$y
  weights <- problem$weights
  if (!is.null(index)) {
    # Sampling is from original observations,
    stopifnot(!problem$grouped)
    z <- z[index]
    y <- y[index]
    weights <- rep(1 / length(index), length(index))
  }
  residual <- dose_response(theta, z) - y
  J <- response_gradient(theta, z)
  as.vector(2 * crossprod(J, weights * residual))
}
```

Computational complexity

One important point to keep in mind:

As datapoints increse, the standard error of the estimate of H decreases with $N^{-1/2}$, however the complexity is increasing by N.
Or stated in more economical terms:
"as we invest more computation time by using more data we get diminishing returns"[Hansen and Larsson, 2026].

Computational complexity

Let N be the sample size, $p = 4$ the number of parameters, and b the batch size.

$$g_B(\theta) = \frac{2}{b} \sum_{i \in B} r_i J_i \quad \Rightarrow \quad O(bp).$$

Method	Gradient cost per update	Additional work
Full-batch GD	$O(Np)$	Parameter update: $O(p)$
Mini-batch SGD	$O(bp)$	Parameter update: $O(p)$
Polyak / Nesterov	Depends on gradient	Momentum: $O(p)$
AdaGrad	$O(bp)$	Coordinate scaling: $O(p)$

One stochastic epoch is approximately N/b updates, giving $\frac{N}{b} O(bp) = O(Np)$.

Step-size rule	Additional cost	Frequency
Armijo backtracking	$O(q_t N)$	Per iteration
Power-law decay	$O(1)$	Per epoch
AdaGrad	$O(p)$	Per update

Here q_t is the number of trial steps.

Implementation: Choices, tests and construction

From solvers.R

```
stochastic_fit <- function {...
    for (epoch in seq_len(epochs)) {
    permutation <- sample.int(N)
    rate <- learning_rate(epoch - 1, gamma0, half_epochs, a)
        for (batch in seq_along(batch_starts)) {
        ...
          if (method == "nesterov") {
            point <- theta + momentum * (theta - theta_previous)}
          g <- gradient(point, problem, index)
          if (method == "sgd") {
            theta[free] <- theta[free] - rate * g[free]}
          if (method == "polyak") {
            rho[free] <- momentum * rho[free] + (1 - momentum) * g[free]
            theta[free] <- theta[free] - rate * rho[free]}
          if (method == "nesterov") {
            theta_previous <- theta
            theta[free] <- point[free] - rate * g[free]}
          if (method == "adagrad") {
            squared_gradients[free] <- squared_gradients[free] + g[free]^2
            theta[free] <- theta[free] - gamma0 * g[free] /
              (sqrt(squared_gradients[free] + epsilon))} ...
}
```

Implementation: Choices, tests and construction

Deterministic algorithms are implemented in with similar structure as the stochastic algorithmns.

We inspect correctness of our algorithms via a series of tests:

From **test-core.R**

```
    numerical_gradient[j] <-
      (objective(plus, problem) - objective(minus, problem)) / (2 * h)
    numerical_hessian[, j] <-
      (gradient(plus, problem) - gradient(minus, problem)) / (2 * h)}

expect_equal(gradient(theta, problem), numerical_gradient, tolerance = 1e-7)
expect_equal(unname(hessian(theta, problem)), numerical_hessian, tolerance =1e-7)
```

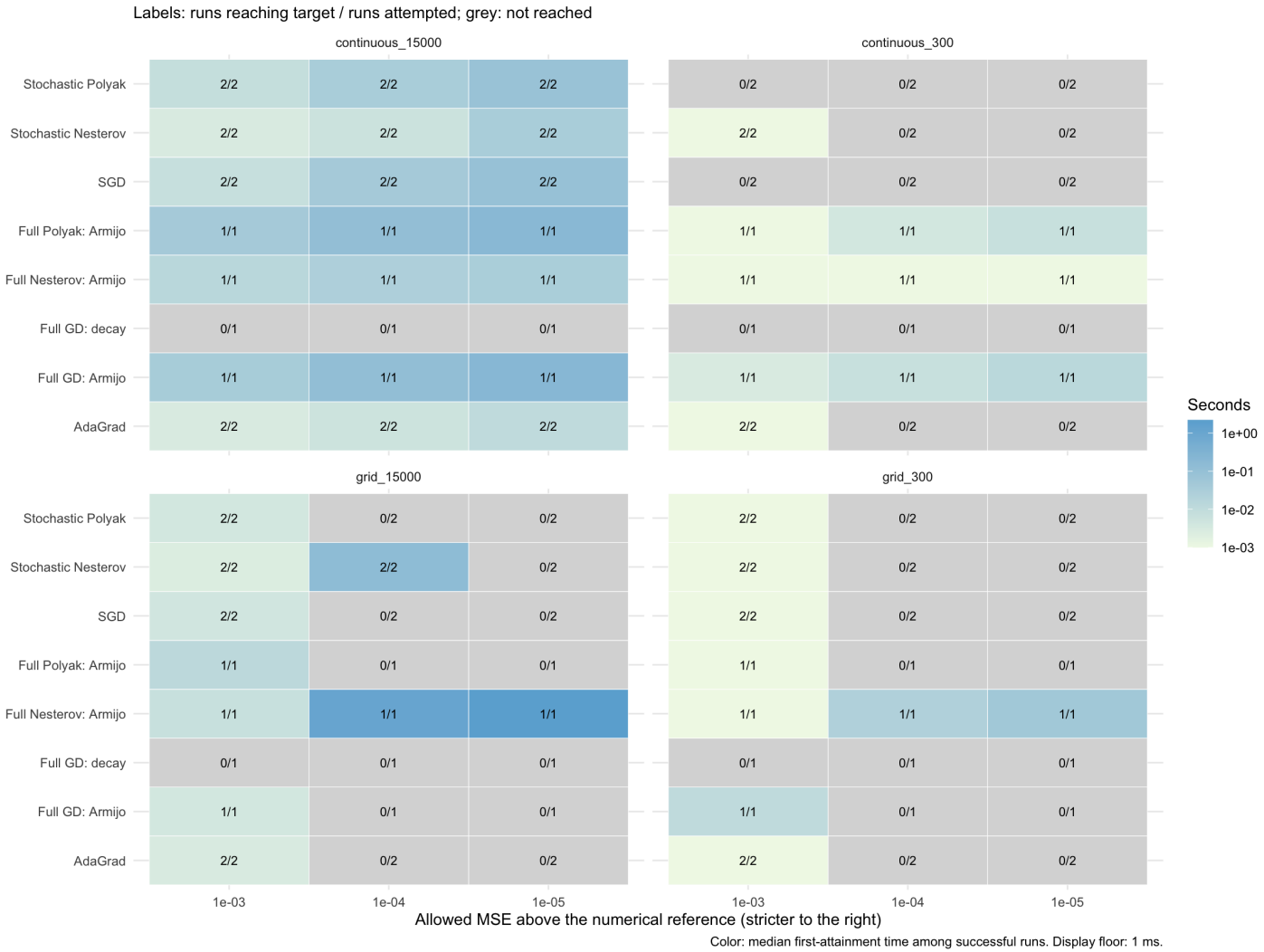
Implementation: Choices, tests and construction

From **test-core.R**

```
test_that("Full-gradient methods agree with nls", {
  problem <- simulate_doses(
    500, "continuous", seed = 42
  )$problem
  reference <- nls(
    y ~ gamma + (delta - gamma) * plogis(alpha - beta * z),
    data = data.frame(y = problem$y, z = problem$z),
    start = as.list(initial_parameters(problem))
  )
  for (method in c("gd", "polyak", "nesterov")) {
    fit <- deterministic_fit(problem, method, maxit = 5000)
    expect_true(all(diff(fit$history$objective) <= 1e-12))
    expect_equal(
      objective(fit$theta, problem),
      mean(residuals(reference)^2), tolerance = 1e-7)}
})
```

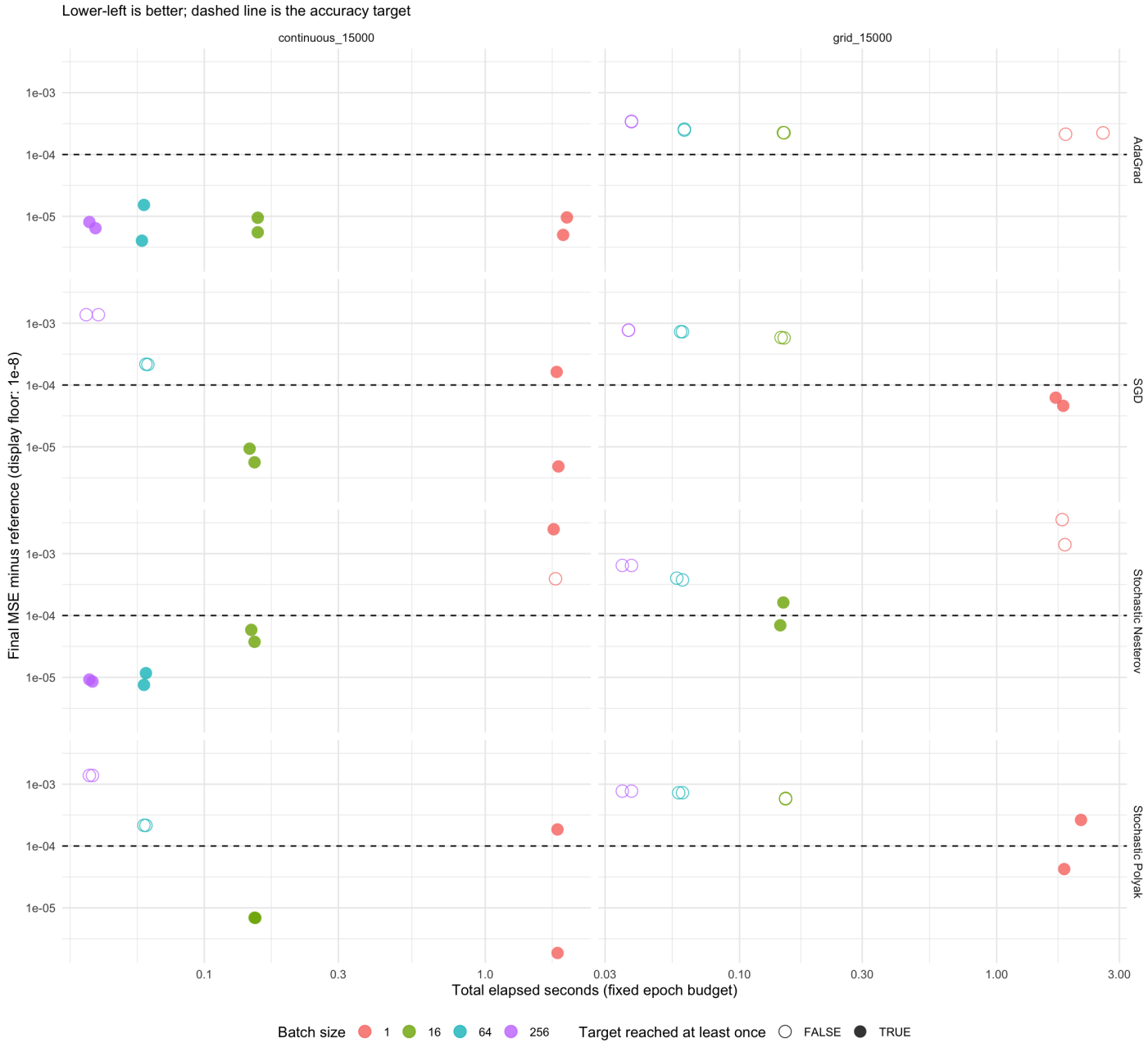
- Accepted steps do not increase the objective, allowing for numerical rounding.
- Final MSE agrees with R's independent `nls()` fit.

Benchmarking and profiling



MSE and time for $N \in \{300, 15000\}$ and x sampled from exponential grid and $\log(x)$ from $\mathcal{N}(0, 2^2)$.

Benchmarking and profiling: Adjusting the batch size



Benchmarking and profiling

We can exploit the data structure if doses are discrete. If n_ℓ data points falls on grid point ℓ, we can ease the gradient computation

$$\nabla H(\theta) = 2 \sum_{\ell=1}^{L} \frac{n_\ell}{N} r_\ell J_\ell,$$

where L denotes the number of observations on each grid point.

N	Method	Objective	Seconds
300	Full GD: Armijo	0.009052	0.077
	Grouped Full GD: Armijo	0.009052	0.029
15000	Full GD: Armijo	0.010232	1.620
	Grouped Full GD: Armijo	0.010232	0.025

Benchmarking and profiling

The comparisons between stochastic and deterministic algorithms hinge on overhead R function calls.

- **Full-gradient descent in R:** Vectorized operations perform much of the work in compiled code.
- **SGD in R:** Looping over data allocation of temporary vectors and Jacobians contribute to overhead, see [Hansen and Larsson, 2026] 11.1.4.
- **SGD in C++:** The entire loop runs in compiled code. Direct data access and reusable storage can reduce indexing, allocation and copying costs.

C++ can reduce the implementation overhead, not the complexity.

Benchmarking and profiling

To display what can be improved through C++, we implement the gradient computation in **gradient.cpp** and access it through `Rcpp`, by `Rcpp::sourceCpp("project_B/gradient.cpp")`.

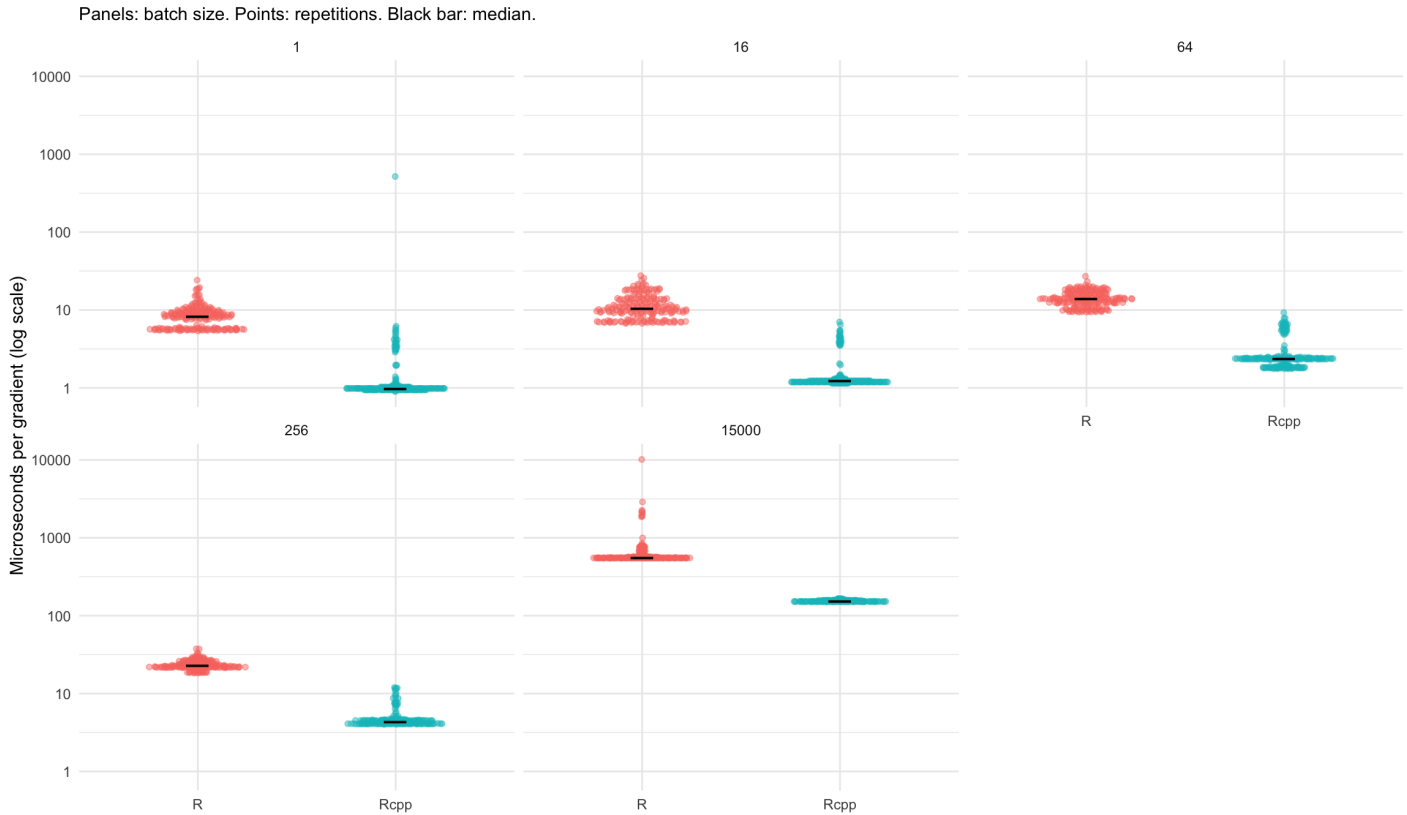
We test the implementation by comparing it to the original gradient implementation.

```
  Rcpp::sourceCpp("../../project_B/gradient.cpp")
  problem <- simulate_doses(100, "continuous", seed = 12)$problem
  theta <- c(0.4, 0.8, 0.1, 1.4)
  index <- c(2L, 5L, 9L)

  r_gradient <- gradient(theta, problem, index)
  cpp_gradient <- dose_gradient_cpp(theta, problem$z, problem$y, inc
  expect_equal(cpp_gradient, r_gradient, tolerance = 1e-10)
})
```

Benchmarking and profiling

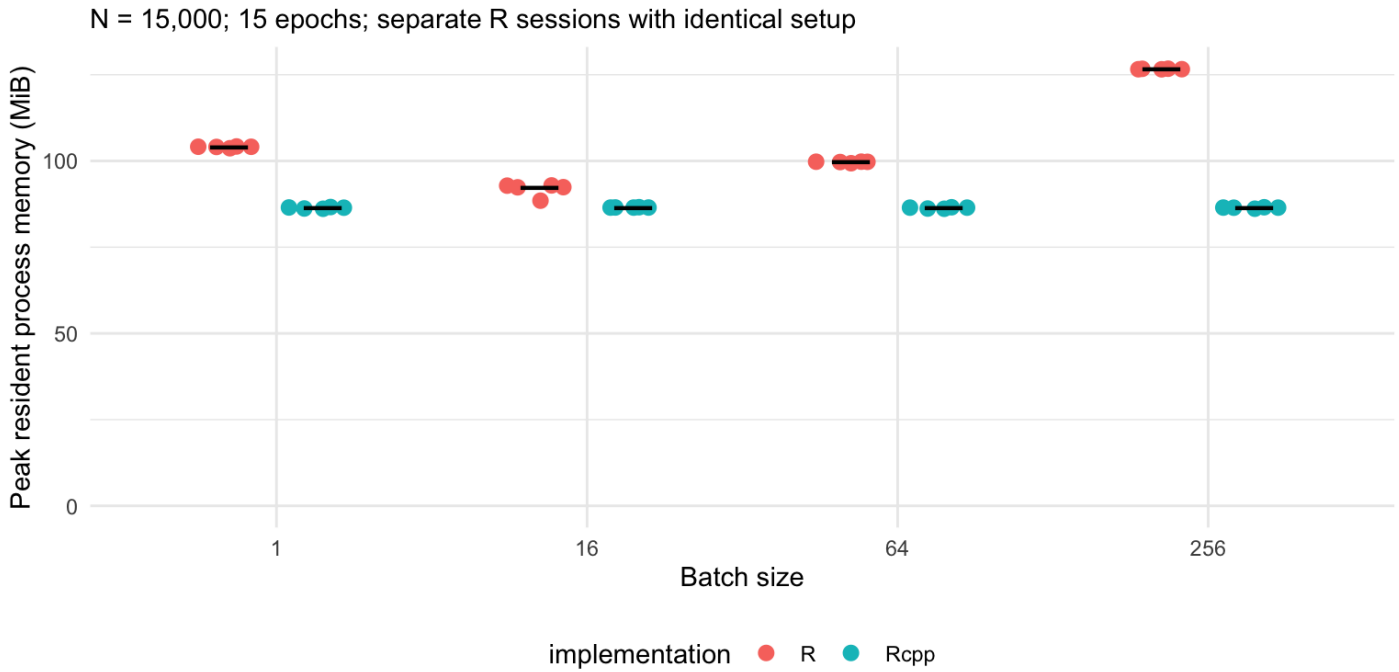
We can improve speed by limiting overhead:



Beeswarm plot for gradient computations at different batch sizes.

Benchmarking and profiling

We can also improve memory allocation of the algorithms



Beeswarm plot for memory uses in SGD across batch sizes $|B| \in \{1, 16, 64, 256\}$

Discussion of choices and potential improvements

The tests performed only show the tip of the iceberg in terms of numerical and computational aspects of stochastic optimization. However these serve as a general exhibition of the most fundamental differences of the algorithms and their computational aspects.
Further studies of the current framework could include:

- Exploring the meaning of initial guess: Convergence metrics vs euclidean norm from $\theta_{\texttt{init}}$ to true parameter θ_0
- Analyzing the relationship between epochs, bach size, runtime and accuracy further
- A broader exploration of θ_0 configurations
- Calibration of leaning rates

Potential performance improvements could consist of exploring:

- Vectorization
- Algorithmic optimization
- Transportation of computations to machine code.

Further performance improvements

1. **Reduce temporary allocations**
   Compute the four gradient components directly, without constructing the Jacobian matrix or a weight vector.

2. **Reduce monitoring frequency**
   Evaluate the full objective less often. This saves computation but gives less precise information about when an accuracy target is reached.

References I



Hansen, N. R. and Larsson, J. (2026).
Computational Statistics with R.
Online book, accessed 2026-10-14.